

Nexus OS Simulator

Advanced Operating System Architecture & Implementation

Project Overview

Nexus OS is a high-performance multitasking operating system simulation developed in C++ using POSIX system calls. The project simulates a kernel environment managing hardware constraints including RAM, HDD storage, and CPU cores. It utilizes real process management techniques found in modern Linux distributions.

System Architecture

- **Kernel Layer:** Handles resource allocation, process lifecycle, and PCB management.
- **Scheduling Layer:** Implements a Multilevel Queue Scheduler with Round Robin and FCFS.
- **Application Layer:** Manages 20+ independent tasks executed via `fork()` and `exec()`.

Core Concepts Implemented

1. Process Management

The OS creates real child processes using `fork()` and replaces the process image with application code using `exec()`. Every application runs in its own isolated xterm terminal window.

2. Resource Synchronization

To prevent race conditions and manage hardware limits, the system uses:

- **Mutex Locks:** Protects the shared Process Control Block (PCB) table.

- **Semaphores:** Limits concurrent execution based on available *N_{cores}*.
- **Condition Variables:** Manages process waiting states when RAM is full.

3. Inter-Process Communication (IPC)

Standard POSIX Pipes are used for communication between the child tasks and the parent OS to request resources dynamically during runtime.

Application Catalog

Category	Modules / Features
File Management	Creator, Editor, Move, Copy, Delete, Info
System Utilities	Clock, Calendar, RAM Viewer, Process Monitor
Interactive	Calculator, Notepad (Multithreaded), Mini-Games

Execution Guide

The system is initialized via terminal with hardware parameters:

```
./OS [RAM_GB] [HDD_GB] [CORES]  
Example: ./OS 4 500 8
```